Designing an AI-Assisted Farm Memory and Agricultural Guidance System for Smallholder Farmers

Doan Trong Luc
Faculty of Software Engineering
FPT University
Da Nang City, Vietnam
lucdtse173180@fpt.edu.vn

Vo Ha Dong
Faculty of Software Engineering
FPT University
Da Nang City, Vietnam
dongvhse@fpt.edu.vn

Nguyen Vu Hoang
Faculty of Software Engineering
FPT University
Da Nang City, Vietnam
hoangnvse@fpt.edu.vn

Le Bao Nhi
Faculty of Software Engineering
FPT University
Da Nang City, Vietnam
nhibtse@fpt.edu.vn

***Abstract*—Smallholder farmers in Vietnam face persistent information asymmetry, limited access to agronomic expertise, and inadequate tools for recording and reusing their own farm history. Existing digital agricultural platforms typically deliver generic, context-free recommendations that do not account for the individual conditions of a given farm. This paper presents FarmDiaries AI, a Progressive Web Application (PWA) that addresses these gaps through three coordinated AI subsystems: (1) a Retrieval-Augmented Generation (RAG) conversational assistant that grounds responses in verified agronomic knowledge combined with the user's personal diary history; (2) a computer vision pipeline using Google Gemini Vision for real-time crop disease diagnosis from smartphone photographs; and (3) an automated weekly insight generator that synthesizes farm activity into actionable weekly recommendations. The proposed system uses NestJS, MongoDB Atlas as the primary datastore, pgvector as a rebuildable vector-search index, Redis-based rate limiting, and BullMQ for background processing. A gamification layer is designed to encourage consistent diary use. This paper describes the system architecture, the Prompt-as-Code methodology, the safety controls, and a planned evaluation framework covering functional verification, retrieval quality, response quality, plant-disease analysis, and usability. The work is presented as a cost-constrained prototype architecture; empirical effectiveness and production readiness remain subjects for future evaluation.**

***Index Terms*—Agricultural AI, Retrieval-Augmented Generation, Crop Disease Detection, Personalized Recommendation, Progressive Web Application, Prompt Engineering, Gemini Vision, pgvector, Smallholder Farming, Vietnam**

I. Introduction

A. Background

Agriculture accounts for roughly 40% of the labor force in rural Vietnam, with the majority of practitioners operating as smallholders cultivating parcels under two hectares across diverse crops including rice (*lúa*), citrus (*bưởi*), coffee (*cà phê*), and vegetables [1]. Smartphone penetration in agricultural provinces exceeded 70% by 2024, creating a direct delivery channel for digital decision-support tools. However, realizing that potential requires software designed around rural constraints: low-bandwidth connectivity, limited formal literacy, preference for conversational Vietnamese, and workflows embedded in daily farm practice rather than periodic desktop sessions.

B. Problem Statement

Three interconnected problems define the target gap.

Knowledge access. Agricultural expertise in Vietnam is concentrated in institutional sources—university extension services, government departments, and printed manuals—that are geographically constrained and temporally infrequent. Existing agricultural chatbots provide population-level knowledge without differentiation for a specific farm's crop varieties, soil history, or applied interventions.

Farm memory. Effective farm management depends on operational history: which pesticide was applied and when, what growth stage a crop reached by a given date, which fields showed disease pressure in the prior season. This knowledge is almost universally held in working memory or handwritten notebooks that resist search and synthesis. Existing diary tools record events but extract no actionable intelligence from the accumulated record.

Disease identification. Plant disease diagnosis is a specialized visual task for which most farmers receive no training. Incorrect identification leads to wrong pesticide selection, crop loss, and consumer safety risks from chemical residue. Field-condition photographs present challenges—variable lighting, occlusion, early-stage symptoms—that poorly-generalized models fail to handle reliably.

C. Research Objectives

This work pursues five objectives:

1) Design a modular AI infrastructure separating LLM invocation, prompt construction, vector retrieval, and embedding generation into independently testable units.
2) Develop a Hybrid RAG architecture combining domain-specific agricultural knowledge with per-user diary memory.
3) Implement a crop disease detection pipeline using Gemini Vision with an upstream image validation layer.
4) Engineer a deterministic, version-controlled prompt construction subsystem with layered injection defense.
5) Evaluate the system on retrieval quality, response accuracy, and user satisfaction metrics.

D. Contributions

The primary contributions are:

- A **Hybrid RAG architecture** that merges a curated agricultural knowledge base with dynamically embedded user diary entries.
- A **Prompt-as-Code approach** (PromptService) isolating prompt construction as a pure, deterministic, testable function with explicit version tracking.
- An **image validation pipeline** applying perceptual hashing, Laplacian blur detection, and magic-bytes verification before LLM Vision invocation.
- A **cost-constrained AI infrastructure** design using provider quotas, Redis-based admission control, and BullMQ scheduling, without assuming that free-tier limits or pricing remain constant.
- An **opt-in community data contribution workflow** (Farm Snap) through which users may explicitly consent to the use of de-identified, quality-reviewed photographs for future model improvement.

II. Related Work

A. Digital Farming Systems

Commercial farm management platforms such as Climate FieldView and AgriWebb deliver robust decision support for large-scale operations but presuppose internet-connected workstations, formal accounting literacy, and subscription economics (10–50 per hectare per year) that exclude smallholders in the Global South [3]. Vietnamese-specific platforms including eFarm (VNPT) and MARD information portals provide valuable advisory content but lack personalization, enforce rigid data-entry formats misaligned with farmer workflows, and integrate no conversational AI capability.

B. AI-Based Agricultural Assistants

Rule-based systems such as Kisan Suvidha (India) and iCow (Kenya) represent first-generation approaches with prescripted response sets. More recent deployments leverage large language models: Pinto et al. [11] adapted GPT-4 for Brazilian soybean advisory, and the AgroPilot project applied Claude-based recommendations for European smallholders. A consistent limitation across these systems is the absence of personal farm context—responses are generated without access to the querying farmer's operational history, which FarmDiaries AI directly addresses through Hybrid RAG.

C. Retrieval-Augmented Generation

Lewis et al. [4] formally characterized RAG as augmenting LLM generation with retrieved text passages to reduce hallucination and improve factual grounding. Sharma et al. [10] demonstrated a 31% improvement in factual accuracy for RAG versus vanilla LLM prompting on agricultural advisory tasks evaluated against agronomist ground truth. FarmDiaries AI extends standard single-corpus RAG with a personalization dimension by simultaneously retrieving from two distinct corpora: a curated knowledge base and the user's personal diary.

D. Plant Disease Detection

Hughes and Salathé [6] introduced the PlantVillage dataset (54,306 labeled images, 26 diseases, 14 crops), enabling Mohanty et al. [5] to achieve 99.35% classification accuracy under controlled laboratory conditions—a figure that degrades substantially under field conditions due to domain shift. Singh et al. [7] (PlantDoc) reported 66.5% accuracy for state-of-the-art models on 13 classes under field conditions, quantifying the domain-shift challenge. EfficientNet [8] and MobileNetV3 [9] architectures have since improved field accuracy while maintaining mobile-deployable inference speeds. FarmDiaries AI uses Gemini Vision for MVP zero-shot diagnosis, with fine-tuned local models planned as the Farm Snap dataset accumulates.

III. Proposed Method

This section presents the FarmDiaries AI methodology as a unified personalized agricultural advisory framework. We first formalize the problem, then describe the three novel contributions that distinguish this work from prior systems: Hybrid RAG with dual-corpus retrieval, Prompt-as-Code engineering, and a cost-constrained AI infrastructure design.

A. Problem Formulation

1) Farm Diary Model: Let $\mathcal{U}$ denote the set of registered farmers. Each farmer $u \in \mathcal{U}$ maintains a personal diary $\mathcal{D}_u = \{d_1, d_2, \ldots, d_n\}$ ordered by timestamp, where each entry d_i is a structured tuple:

$$d_i = \langle c_i,\ g_i,\ \tau_i,\ t_i,\ w_i,\ \mathbf{P}_i \rangle \tag{1}$$

in which $c_i \in \mathcal{C}$ is the crop type (e.g., *lúa*, *bắp*), g_i is the growth stage, τ_i is the UTC timestamp, t_i is the free-text note, w_i is the weather context (temperature, humidity, rainfall), and $\mathbf{P}_i$ is the set of associated photographs.

2) Three Advisory Sub-Problems: Prior work [10], [11] treats agricultural advisory as a single-corpus retrieval task over a shared knowledge base $\mathcal{K}$, producing responses that are identical for all farmers querying the same question. FarmDiaries AI decomposes the problem into three sub-problems that jointly require *both* domain knowledge and personal farm context:

P1 — Personalized Advisory (PA). Given query q from farmer u, generate response r grounded in $\mathcal{K}$ *and* $\mathcal{D}_u$:

$$r = f_{\text{LLM}}\big(q,\ \mathcal{C}_{\text{hybrid}}(q, u)\big) \tag{2}$$

P2 — Visual Disease Detection (DD). Given photograph $p \in \mathbf{P}_i$, predict structured diagnosis:

$$\hat{y} = f_{\text{vision}}(V(p)) \tag{3}$$

where $V(\cdot)$ denotes the upstream validation pipeline (Section III-D).

P3 — Weekly Insight Synthesis (IS). Given the seven-day diary window $\mathcal{D}_u^{(7)} = \{d_i \mid \tau_i \geq \tau_{\text{now}} - 7\text{d}\}$, generate a forward-looking farm report:

$$s_u = f_{\text{LLM}}\big(\mathcal{C}_{\text{hybrid}}(\mathcal{D}_u^{(7)}, u)\big) \tag{4}$$

The key distinction from prior work is the shared context function $\mathcal{C}_{\text{hybrid}}$, which retrieves from *two* corpora simultaneously. This is described in Section III-B.

B. Hybrid RAG Architecture

1) Motivation: Standard RAG [4] retrieves from a single corpus $\mathcal{K}$, producing population-level responses. Sharma et al. [10] demonstrated a 31% accuracy improvement over vanilla LLM prompting, but their system still lacks per-farmer context. FarmDiaries AI introduces **Hybrid RAG**: simultaneous dual-corpus retrieval over $\mathcal{K}$ (verified agronomic knowledge) and $\mathcal{D}_u$ (personal diary history), enabling responses such as: *"Brown planthopper was treated 11 days ago on your field; given current humidity, re-application may be warranted."* — a response impossible without the personal memory dimension.

2) Retrieval Algorithm: Let $\phi : \Sigma^* \to \mathbb{R}^{768}$ denote the `text-embedding-004` embedding function. The context assembly procedure $\mathcal{C}_{\text{hybrid}}$ proceeds as follows:

1) Compute query embedding: $\mathbf{e}_q = \phi(q) \in \mathbb{R}^{768}$.
2) Retrieve top-k candidates from each corpus via Approximate Nearest Neighbour (ANN) search with cosine similarity $\text{sim}(\cdot,\cdot)$, minimum threshold $\theta = 0.5$:

$$R_{\mathcal{K}} = \text{ANN}(\mathbf{e}_q,\ \mathcal{K},\ k{=}10,\ \theta) \tag{5}$$

$$R_{\mathcal{D}} = \text{ANN}(\mathbf{e}_q,\ \mathcal{D}_u,\ k{=}10,\ \theta) \tag{6}$$

3) Merge and rank by descending similarity score:

$$R = \text{sort}_{\downarrow}\big(R_{\mathcal{K}} \cup R_{\mathcal{D}},\ \text{sim}\big) \tag{7}$$

4) Fetch full document content from MongoDB using $\{\, r.\texttt{source\_id} \mid r \in R \,\}$. *Note: pgvector stores only vector indices; all text content resides in MongoDB.*
5) Assemble context string $\mathcal{C}$ under budget $B = 6{,}000$ characters, ordered by descending score.

ANN search uses a pgvector HNSW index with initial parameters $m{=}16$ and ef_construction=64. These values are configuration defaults rather than verified performance guarantees and must be benchmarked on the deployed dataset and infrastructure. Only rows with `is_active = TRUE` are considered. Diary-memory retrieval additionally enforces the authenticated-user invariant `metadata.userId = authenticatedUserId`; this filter is applied by application code and cannot be supplied or overridden by the model.

3) Text Chunking Policy: Raw diary notes are segmented before embedding. Let $\ell(\cdot)$ denote character length after whitespace normalization:

$$\text{chunk}(t) = \begin{cases} \emptyset & \ell(t) < 20 \\ \{\, t \,\} & 20 \leq \ell(t) < 500 \\ \text{slide}(t,\ w{=}300,\ s{=}100,\ m{=}10) & \ell(t) \geq 500 \end{cases} \tag{8}$$

where $\text{slide}(t, w, s, m)$ produces up to m overlapping windows of width w and step s. Knowledge documents use $w{=}500$, $s{=}150$, $m{=}50$ to preserve argumentative context across longer passages. The 20-character floor eliminates semantically vacuous entries (e.g., single-word notes) that would add noise to the index.

4) Embedding Lifecycle: Embeddings are written asynchronously via BullMQ to decouple latency-sensitive diary creation from the embedding API call. The **Dual Write** invariant is:

$$\forall\, d_i \in \mathcal{D}_u :\ d_i \in \text{MongoDB} \implies \exists\ \text{BullMQ job for } \phi(d_i) \tag{9}$$

If the embedding job fails permanently after 3 exponential-backoff retries, d_i remains safely in MongoDB; only its search coverage is temporarily reduced. A nightly gap-fill cron requeues any entries without active embeddings, bounding the maximum staleness window to 24 hours.

On diary update, the replacement embeddings are generated first and activated atomically with deactivation of the previous rows where supported by the index store. If generation fails, the previous active vectors remain available. Inactive rows are removed later by a cleanup job. This ordering avoids a temporary loss of retrieval coverage.

C. Prompt-as-Code Engineering

1) Design Rationale: Existing LLM-based agricultural systems embed prompt strings directly in business logic, making them untestable and prone to silent regressions when templates change [11]. FarmDiaries AI introduces **Prompt-as-Code**: prompt construction is isolated in a dedicated `PromptModule` that accepts structured inputs and returns a versioned `BuiltPrompt` record:

$$\texttt{BuiltPrompt} = \langle\, \pi_{\text{sys}},\ \pi_{\text{usr}},\ v,\ \tau_{\text{build}} \,\rangle \tag{10}$$

where π_{sys} is the system prompt, π_{usr} is the user-turn content, v is the semantic version string, and τ_{build} is the build timestamp logged for regression detection.

2) Determinism Guarantee: Given identical inputs $(q, \mathcal{C}, h, m_{\text{pet}})$ — query, context, conversation history, and pet mood — the builder produces byte-identical output. Non-deterministic primitives (`Date.now()`, `Math.random()`) are *prohibited* inside builder functions. This guarantee enables snapshot-based unit testing. The planned suite covers all three builder methods (`buildChatPrompt`, `buildVisionPrompt`, and

`buildWeeklyInsightPrompt`); test counts and coverage percentages are reported only after they are produced by the implemented codebase and CI pipeline.

3) Layered Injection Defense: User-supplied and retrieved text are treated as untrusted data. Two sanitization functions provide a supplementary filtering layer before structured prompt assembly:

- σ_{msg}: applied to user messages. Blocklist includes `[SYSTEM]`, `[INST]`, "ignore previous instructions", "act as", and LLM special tokens <| . . . |>.
- σ_{ctx}: applied to RAG-retrieved context, independently extensible from σ_{msg} to handle adversarial document injection.

Separating the two sanitizers allows targeted hardening of each attack surface. The blocklists are not treated as a complete injection defense: retrieved passages are placed in explicitly delimited data fields, are not permitted to define system policy or tool calls, and model outputs are validated against application-owned schemas and authorization rules.

4) Slot Truncation: Hard character limits prevent context-window overflow while preserving the highest-relevance content:

TABLE I
PROMPT SLOT TRUNCATION LIMITS

Slot	Char Limit
User message	2 000
RAG context	6 000
Conversation history	4 000 (last 6 turns)

D. Visual Disease Detection Pipeline

1) Upstream Validation: To minimize Gemini Vision API calls and enforce minimum image quality, four sequential gates are applied before model invocation (Figure 2). Gate 1 enforces per-user daily quotas via Redis counters (3 scans/day free; 10 scans/day premium). Gate 2 inspects binary magic bytes using the `file-type` library, independently of the client-declared MIME type, to block format spoofing. Gate 3 computes the Laplacian variance σ_L^2 via Sharp.js and rejects images with $\sigma_L^2 < 100$, calibrated empirically on field images rated unusable by agronomist reviewers. Gate 4 checks perceptual hash (pHash) similarity: images within Hamming distance $\delta_H < 10$ of a cached scan within 7 days return the cached result, eliminating duplicate API calls.

2) Structured Output Contract: The Gemini Vision prompt enforces a strict JSON output schema:

$$\hat{y} = \langle\, \mathit{is\_plant},\ \mathit{disease},\ \mathit{certainty},\ \mathit{evidence},\ \mathit{symptoms},\ \mathit{treatment\_hint},\ \mathit{disc} \,\rangle \tag{11}$$

where the diagnostic output includes a categorical *diagnostic certainty* value (`low`, `medium`, or `high`), an *evidence quality* object describing visible symptoms and image limitations, a PHI-related warning field, and a disclaimer. These fields represent model-reported evidence rather than calibrated probabilities. Low certainty or insufficient evidence forces a retake or expert-consultation warning. PHI information is never assumed correct merely because it is present in model output.

3) Post-Processing Safety Guardrail: A deterministic post-processing function cross-references `treatment.chemical` against the Vietnamese Ministry of Agriculture BVTV restricted pesticide registry. Any match appends an explicit safety alert. This check is enforced in application code rather than delegated to the model. It reduces risk but does not establish complete safety compliance; registry freshness, entity matching, dosage interpretation, and human review remain necessary.

E. Cost-Constrained AI Infrastructure

A core design goal of FarmDiaries AI is to minimize infrastructure expenditure during the MVP phase. The architecture uses free or low-cost service tiers where available, but does not assume that quotas, prices, service-level guarantees, or eligibility remain unchanged. Three mechanisms support this cost-constrained deployment.

1) Gemini Free-Tier Quota Management: LLMModule maintains provider-specific Redis counters using configurable limits loaded from deployment settings. When the configured generation quota is saturated, interactive chat returns a graceful fallback, while background jobs receive a rate-limit exception and retry with bounded exponential backoff. Concrete quota values are verified against the provider documentation at deployment time rather than hard-coded as research claims.

2) pgvector over Atlas Vector Search: pgvector with HNSW was selected as a separately deployable, rebuildable search index because it provides explicit control over indexing and can be operated independently from the MongoDB business datastore. No fixed latency or price advantage is assumed without a deployment-specific benchmark and a current provider-cost review. The separation of vector index (pgvector) from document store (MongoDB) preserves recoverability: if the pgvector index is lost, it can be fully reconstructed by re-running the embedding pipeline over MongoDB.

3) Delay Spreading for Batch Insight Jobs: The weekly insight generator must process all N users within a Sunday morning window without exceeding the 15 RPM free-tier quota. A **Delay Spreading Algorithm** assigns each user u_i a BullMQ job delay:

$$\Delta_i = \left\lfloor \frac{i}{N} \times T_{\text{win}} \right\rfloor, \quad T_{\text{win}} = 14{,}400\,\text{s} \tag{12}$$

This distributes jobs uniformly over four hours. It reduces burstiness, but does not by itself guarantee quota compliance for arbitrary N; worker concurrency and a token-bucket admission controller enforce the actual provider limit.

F. Gamification as Behavioral Data Collection

Consistent diary logging is both a user engagement goal and a prerequisite for high-quality RAG personalization: sparse diaries yield sparse context. The virtual pet (Bé Thóc) provides

behavioral reinforcement by exposing a deterministic mood function:

$$m(u) = \begin{cases} \text{excited} & \text{if } \sigma_u \in \{7,\ 14,\ 30\} \\ \text{happy} & \text{if } \rho_u \leq 24\,\text{h} \\ \text{neutral} & \text{if } 24 < \rho_u \leq 36\,\text{h} \\ \text{sad} & \text{if } \rho_u > 36\,\text{h} \\ \text{worried} & \text{if active disease detected} \end{cases} \tag{13}$$

where σ_u is the current diary streak (days) and ρ_u is hours elapsed since the last entry. The mood value $m(u)$ is injected into the chat system prompt via `buildChatPrompt`, enabling tone adaptation and streak-milestone celebrations. This creates a self-reinforcing loop: consistent logging $\rightarrow$ richer RAG context $\rightarrow$ higher-quality personalized responses $\rightarrow$ increased user engagement.

The Farm Snap community photo-sharing feature may extend this loop into a voluntary data contribution workflow. A submission is eligible for research or model-improvement use only when the user explicitly opts in. Precise coordinates are excluded or coarsened unless they are required and separately consented to; exported samples are de-identified, quality-reviewed, and removable when consent is withdrawn.

IV. System Architecture

A. Overview

FarmDiaries AI follows a Decoupled Modern Web PWA / Hybrid Cloud architecture. The React Vite PWA client communicates with the NestJS backend exclusively over HTTPS REST. The backend implements Hexagonal Architecture across four layers: Gateway (controllers, webhooks), Application (guards, pipes, interceptors), Domain (business services, AI pipeline), and Infrastructure (MongoDB, pgvector, Redis, BullMQ, Cloudflare R2, Gemini API). Figure 1 illustrates the high-level topology.

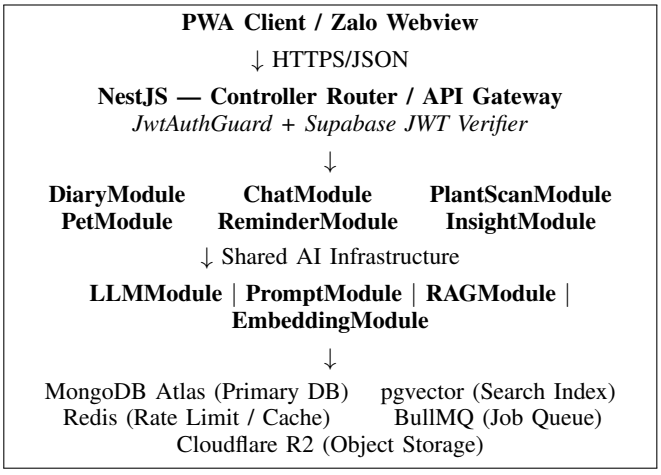

Fig. 1. FarmDiaries AI high-level system architecture.

B. Database Layer

MongoDB Atlas is the **exclusive primary database** for all application data: user profiles, diary entries, chat sessions, pet states, plant scan records, weekly insights, reminders, audit logs, and knowledge content. pgvector (a PostgreSQL extension) serves solely as a **vector search index**, containing one table (`embeddings`) with columns `(id, source_id, source_type, embedding vector(768), metadata, is_active, created_at)` and no business data.

This separation provides two properties: (1) if the pgvector index is lost, it can be fully reconstructed from MongoDB by re-running the embedding pipeline; (2) all CRUD operations in application code target MongoDB, preserving a single coherent source of truth.

pgvector was selected as a rebuildable search-index component after considering operational control and the desire to keep business records exclusively in MongoDB. Latency and total cost must be measured against current service offerings and the actual corpus before deployment; therefore, the architecture does not rely on fixed provider-pricing or latency claims.

C. Caching and Queue Architecture

Redis serves three roles: (1) rate-limit counters for Gemini API quota enforcement; (2) plant scan result caching keyed on perceptual hash; and (3) BullMQ broker state. BullMQ manages four priority queues: `llm_queue` (priority 1 for interactive chat, priority 2 for plant scans), `embed_queue` (priority 3), `reminder_queue` (priority 5), and `insight_queue` (priority 10, lowest). This hierarchy ensures interactive requests are never blocked by background batch operations.

D. Object Storage and File Security

Cloudflare R2 (S3-compatible API) hosts all binary assets. The backend issues short-lived pre-signed GET URLs (TTL = 1 hour) for client download and short-lived signed PUT URLs (TTL = 5 minutes) for client upload. File uploads pass two independent validation checks: MIME type filtering via Multer `fileFilter`, and magic-bytes verification using the `file-type` library to detect format spoofing.

E. Notification Architecture

The notification subsystem supports a configurable delivery chain: in-app/PWA notification, optional Zalo template messages, and optional email fallback. Channel priority depends on user consent, account configuration, deliverability, and verified provider capabilities. Five notification categories are planned: diary reminder, water reminder, weekly insight delivery, streak milestone, and plant disease alert.

V. AI Architecture

FarmDiaries AI separates AI concerns across four modules with well-defined interfaces, enabling independent development and testing.

A. LLMModule

LLMModule is the **singleton gateway** for all Gemini API calls. No other module invokes Gemini directly. The module exposes three methods: `complete()` for text generation via Gemini Flash, `completeVision()` for multimodal diagnosis, and `embed()` for 768-dimensional vector generation via `text-embedding-004`.

Rate limiting uses independent Redis counters or token buckets per provider operation. Limits are deployment configuration derived from current provider quotas. When the generation limit is saturated, interactive chat returns a fallback message and background jobs receive a rate-limit exception. Retry logic uses bounded exponential backoff before invoking the fallback path.

B. PromptModule

PromptModule implements a **Prompt-as-Code** approach: prompt construction is a pure, side-effect-free service accepting structured input and returning a versioned `BuiltPrompt` object. Three builder methods correspond to the three AI pipelines: `buildChatPrompt()`, `buildVisionPrompt()`, and `buildWeeklyInsightPrompt()`.

Determinism requirement: given identical inputs, builders produce byte-identical output. No internal use of `Date.now()`, `Math.random()`, or other non-deterministic operations is permitted.

Injection resistance: user messages and retrieved context are treated as untrusted data and inserted only into delimited prompt fields. Separate sanitization functions may remove known control-token patterns, but blocklists are supplementary rather than authoritative. Application code owns authorization, tool permissions, citation provenance, and output-schema validation; retrieved text cannot modify those controls.

Truncation: character limits are applied independently—user messages: 2,000 chars; RAG context: 6,000 chars; conversation history: 4,000 chars retaining the most recent six turns.

The `promptVersion` field in `BuiltPrompt` is passed to LLMModule for logging, enabling regression detection when templates change.

C. RAGModule

RAGModule implements context retrieval for Chat AI and Weekly Insight pipelines. The retrieval process is:

1) Embed the user query via `text-embedding-004` ($\mathbb{R}^{768}$).
2) Query the pgvector HNSW index using cosine similarity and application-configured candidate limits. Knowledge retrieval filters by source status and optional crop metadata. Diary retrieval additionally applies the non-bypassable filter `metadata.userId = authenticatedUserId`.
3) Receive `(source_id, source_type, chunk_id, score)` tuples only—no business content from pgvector.
4) Fetch authorized full documents from MongoDB and discard missing, deleted, or unauthorized records.
5) Optionally combine semantic score with lexical, crop, source-authority, and recency signals before assembling context within a configured budget.
6) Build citation provenance from the retrieved records in application code; the model is not trusted to invent source identifiers.

The Hybrid design simultaneously retrieves from two corpora: knowledge chunks (verified agronomic documents) and diary entries (user-authored records). This allows responses to reference both domain knowledge (e.g., disease biology) and personal farm context (e.g., a specific treatment applied 11 days prior).

Every returned context item carries application-generated provenance: `sourceType`, `sourceId`, `chunkId`, retrieval score, and an authorized display label. Citations shown to the user are rendered from this provenance after generation; free-form source identifiers emitted by the LLM are ignored.

TABLE II
RAG RETRIEVAL EVALUATION TARGETS

Metric	Definition	Target
Precision@6	Proportion of retrieved docs rated relevant by agronomist	≥ 0.70
Recall@6	Proportion of known-relevant docs retrieved	≥ 0.60
Context Relevance	Avg. cosine similarity: context vs. ground-truth answer	≥ 0.75

D. EmbeddingModule

EmbeddingModule manages the lifecycle of vector embeddings in pgvector. Embeddings are created asynchronously via BullMQ jobs, decoupled from the synchronous MongoDB write that precedes them (Dual Write pattern). If an embedding job fails permanently (after 3 retries with exponential backoff), the diary entry is safe in MongoDB—only search coverage is temporarily reduced. A nightly gap-fill cron identifies entries without active embeddings and re-queues their jobs.

Text chunking strategy by source type:

- **Diary notes** < 20 **chars**: skipped—semantically insufficient.
- **Diary notes 20–499 chars**: embedded as a single chunk.
- **Diary notes** ≥ 500 **chars**: sliding window, 300-char window, 100-char step, maximum 10 chunks.
- **Knowledge documents**: 500-char window, 150-char step, maximum 50 chunks per document.

When a diary entry changes, new embeddings are generated before the index pointer is switched. After the replacement rows are active, previous rows are marked inactive and later removed by a cleanup job. If regeneration fails, the previous active rows remain available, preserving retrieval coverage.

VI. Plant Disease Analysis

A. Validation Pipeline

The plant disease detection pipeline applies three sequential validation layers before invoking Gemini Vision, reducing unnecessary API calls and ensuring minimum image quality.

1. Rate Limit Check
Redis counter: 3 scans/day (free) / 10 scans/day (premium)

↓ PASS

2. Magic Bytes Validation
file-type library: must match JPEG / PNG / WebP header

↓ PASS

3. Blur Detection (Sharp.js)
Laplacian variance $< 100 \Rightarrow$ reject, prompt retake

↓ PASS

4. Perceptual Hash (pHash) Cache
Hamming distance < 10 within 7 days $\Rightarrow$ return cached result

↓ CACHE MISS

5. Cloudflare R2 Upload + Gemini Vision
Structured JSON output: diagnosis candidates, certainty, evidence quality, treatment cautions

↓

6. Persist to MongoDB + Return to Client

Fig. 2. Plant disease scan pipeline with upstream validation layers.

Blur detection: the Laplacian variance of the image is computed via an image-processing component. An initial threshold may be used to request a retake, but the value must be calibrated and reported on a labeled field-image validation set before being treated as a validated quality boundary.

Perceptual hash caching: a pHash is computed for each accepted image. Images with Hamming distance below 10 from a cached scan within the past 7 days return the cached diagnosis, eliminating duplicate API calls for the same or visually similar field conditions.

Magic-bytes verification: the binary header of each uploaded file is inspected independently of the declared MIME type to prevent format spoofing.

B. Gemini Vision Prompt

The vision prompt instructs Gemini to return a structured JSON response containing: `isPlant` (boolean), one or more diagnosis candidates, `diagnosticCertainty` (`low|medium|high`), an `evidenceQuality` object, visible symptoms, treatment options, safety cautions, PHI-related information when a specific registered product is identified, and a disclaimer. These values are model assessments, not calibrated probabilities. Low certainty, poor evidence quality, or conflicting candidates require a retake or expert-consultation warning. If `isPlant` is false, the pipeline returns a structured negative without a disease label.

C. Safety Guardrails

A post-processing function applies the BVTV (plant protection) guardrail: any treatment recommendation referencing a restricted or prohibited pesticide under Vietnamese regulation is flagged with an explicit safety alert. All diagnosis responses carry a disclaimer advising consultation with a local agricultural extension officer. This guardrail is implemented in application code rather than relying solely on model output. It is a risk-reduction measure and does not replace verification against current regulations or professional agronomic advice.

D. Future Direction

The MVP Gemini Vision approach provides immediate utility but faces latency (5–10 s round-trip), external API dependency, and potential output format inconsistency. Future work will fine-tune EfficientNet-B4 or MobileNetV3 on a Vietnamese field-condition dataset seeded through the Farm Snap data flywheel. Candidate architectures include:

- **EfficientNet-B4** [8]: strong per-class accuracy on multi-disease classification with compact model size.
- **MobileNetV3** [9]: optimized for mobile inference, enabling potential on-device deployment.
- **YOLO classification** [12]: single-pass real-time classification within a camera viewfinder.

VII. Personalized Farm Memory

A. From Generic to Personalized Advice

Generic agricultural chatbots answer: *"What disease affects rice during the tillering stage?"* FarmDiaries AI answers: *"Given that your rice field was last treated for brown planthopper 11 days ago, soil moisture was low in four of the past seven diary entries, and humidity is elevated this week, what is the most appropriate intervention?"* This requires treating diary history as operational memory addressable by semantic query.

B. Diary as Farm Memory

Each diary entry captures: crop type, growth stage, free-text notes, photographs, watering and fertilization events, weather context, and geographic location. Upon creation, entries are persisted to MongoDB synchronously and enqueued for embedding asynchronously. Once embedded, entries are retrievable via cosine similarity against user queries in the RAG pipeline.

Diary data serves three distinct roles:

1) **UI History Layer**: chronological operational record in the diary list view.
2) **RAG Memory Layer**: embedded chunks retrieved as context for AI chat.
3) **Weekly Insight Source**: the past seven days of entries are synthesized into a weekly farm activity summary with forward-looking recommendations.

C. Pet State as Behavioral Context

The virtual pet mascot (Bé Thóc) maintains a rule-based mood derived from the farmer's diary streak and entry recency. Mood values (`happy`, `excited`, `neutral`, `sad`, `worried`) are injected into the chat system prompt, allowing

the AI to adapt tone and include streak-appropriate encouragement. This creates a feedback loop: AI responses reinforce the consistent diary behavior that in turn enriches AI response quality.

D. Farm Snap Data Flywheel

Farm Snap is an instant photo-sharing feature with a dual purpose. Users experience it as a community feed where they share field photographs and receive peer reactions. A submission becomes eligible for model-improvement research only after explicit, revocable opt-in consent. The user supplies `cropType` and `condition` (healthy / issue / harvest / other). Location is omitted by default or coarsened to the minimum useful granularity; precise coordinates require a separate purpose and consent. Only de-identified, quality-reviewed samples may be exported, and withdrawal workflows must remove future eligibility.

VIII. Implementation

A. Frontend

The client is a PWA implemented in React 18, Vite, and TypeScript. Styling uses Tailwind CSS with Shadcn/UI primitives. State management uses Zustand for local state and TanStack Query for server state with cache invalidation. PWA capabilities (offline caching, installability) are implemented via `vite-plugin-pwa`. The application is mobile-first, targeting portrait-orientation smartphone use.

B. Backend

The backend is built on NestJS 10 with TypeScript strict mode. The module system enforces controller / service / DTO / repository separation within each feature domain. Input validation uses class-validator at the DTO layer with a global ValidationPipe configured for property whitelisting and non-whitelisted rejection. Supabase Auth owns identity and application sessions. The client obtains a Supabase session, and NestJS verifies the Supabase JWT for each protected request using a Passport strategy or equivalent verifier. The backend does not issue a second custom refresh-token family.

C. AI Layer Dependency Hierarchy

The AI module dependency order is:

$$\text{LLMModule} \leftarrow \text{EmbeddingModule} \leftarrow \text{RAGModule} \leftarrow \text{Chat/Scan/Insight} \tag{14}$$

PromptModule has no dependencies, making it independently unit-testable without mocking. LLMModule is at the base; no module above it invokes Gemini directly. This hierarchy prevents scattered API calls, centralizes quota management, and simplifies provider substitution.

D. Infrastructure

Docker Compose orchestrates local development with services for NestJS, MongoDB, PostgreSQL+pgvector, and Redis. A candidate deployment uses an application hosting provider, MongoDB Atlas, a PostgreSQL service with pgvector support, Redis, and Cloudflare R2. Final providers are selected after verifying current quotas, pricing, region availability, and operational requirements. CI/CD runs on GitHub Actions: lint $\rightarrow$ type-check $\rightarrow$ unit tests $\rightarrow$ integration tests (with pgvector, Redis, and MongoDB service containers) $\rightarrow$ build verification $\rightarrow$ staging deploy $\rightarrow$ smoke test $\rightarrow$ production deploy.

TABLE III
Infrastructure Stack

Layer	Technology	Role
Frontend	React 18 + Vite PWA	Client application
Backend	NestJS 10 + TypeScript	API server
Primary DB	MongoDB Atlas	All business data
Vector Index	pgvector (HNSW)	Similarity search only
Cache / Queue	Redis + BullMQ	Rate limit, job scheduling
Storage	Cloudflare R2	Binary assets
Auth	Supabase Auth	Identity provider
LLM	Gemini Flash	Text generation
Embedding	text-embedding-004	768-dim vectors
Vision	Gemini Vision	Crop disease diagnosis
Notification	Zalo ZNS / Web Push	User alerts

IX. Evaluation Methodology

A. Functional Testing

Functional correctness will be evaluated through a three-tier test pyramid. Unit tests target at least 70% line and branch coverage for critical modules, including LLM retry and fallback logic, notification routing, PromptModule builders, plant-scan validators, authorization filters, and RAG context assembly. Exact test counts and achieved coverage will be inserted from CI artifacts after implementation. Integration tests verify the two-phase retrieval pattern (pgvector $\rightarrow$ MongoDB), cross-user isolation, citation provenance, Supabase JWT verification and revocation behavior, and the diary CRUD plus embedding-trigger chain. End-to-end Playwright tests cover the complete diary creation flow, AI chat interaction, plant scan submission, and notification delivery.

B. User Acceptance Testing

The planned User Acceptance Testing (UAT) protocol uses structured sessions with recruited smallholder farmers. Recruitment site, sample size, inclusion criteria, consent procedure, and ethics requirements will be documented before data collection. Each participant completes four standardized scenarios: (1) diary entry creation, (2) AI chat with a crop-specific problem, (3) plant scan submission, and (4) weekly insight review. The System Usability Scale (SUS) is administered post-session; the minimum acceptance threshold is a SUS score of 70 (“Good”).

C. AI Response Evaluation

AI response quality is assessed through a dual evaluation protocol. Automated evaluation may use a separately configured LLM judge for consistency checks, but it is not treated as ground truth. Human agronomist review remains the primary validity source for factual and safety judgments. Human evaluation by qualified agronomists applies 5-point Likert scoring on the same dimensions. Inter-rater reliability is computed via Cohen's kappa.

TABLE IV
AI RESPONSE EVALUATION TARGETS

Metric	Definition	Target
Factual Accuracy	Agreement with agronomic reference	≥ 0.80
Response Relevance	On-topic answer to agricultural query	≥ 0.85
Safety Compliance	PHI warning present when pesticide recommended	1.00
User Satisfaction	In-app 5-star rating	$\geq 4.0/5.0$

D. Plant Disease Detection Evaluation

The proposed disease-detection evaluation uses a preregistered field-image set with crop-disease coverage, sample size, and ground-truth labels established by qualified reviewers. The final paper will report the actual dataset composition rather than assuming 200 images or 10 combinations in advance. Standard precision, recall, and F1 score are computed per-class and overall:

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN} \tag{15}$$

$$F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \tag{16}$$

False negative rate for high-severity diseases receives particular attention, as missed detection carries significant economic consequences. Pipeline-level metrics additionally include: false positive rate of the blur detection filter, pHash cache hit rate, and end-to-end latency distribution.

X. LIMITATIONS

LLM hallucination. Despite RAG grounding, Gemini Flash retains potential to generate plausible but factually incorrect recommendations for crop-disease combinations absent from the knowledge base. Pesticide-related responses are particularly sensitive, as incorrect dosage or PHI information creates consumer safety risk.

Knowledge base coverage. The knowledge base is limited to documents curated by the development team and partner agronomists. Gaps exist for regionally specific varieties, traditional farming practices, and recently emerged disease strains.

External API dependency. Core AI functionality depends on Gemini API availability. Extended outages would degrade chat and disease scan pipelines to non-functional states. LLM-Module's abstraction enables provider substitution, but an alternative provider integration is not implemented in the MVP.

Field image quality. The Gemini Vision pipeline inherits limitations of zero-shot multimodal models for specialized Vietnamese crop varieties underrepresented in Gemini's training distribution. Confidence calibration may not accurately reflect true diagnostic uncertainty.

MVP scope. Deferred features include offline chat, multilingual support beyond Vietnamese, automatic weather API integration for diary enrichment, and longitudinal crop health analytics.

XI. FUTURE WORK

Farm memory expansion. Future RAG retrievals will include plant scan results and weekly insight summaries alongside diary entries, creating a comprehensive personal farm memory spanning all data collected by the application.

Fine-tuned disease detection. When sufficient quality-reviewed Farm Snap samples are accumulated (estimated minimum 5,000 labeled images per major crop category), EfficientNet-B4 or MobileNetV3 fine-tuning is projected to improve per-class precision and recall relative to the zero-shot Gemini Vision baseline, particularly for diseases underrepresented in global training data. On-device inference would enable offline detection capability.

Regional disease knowledge graph. Aggregated anonymized patterns from diary and scan records could support a regional crop health knowledge graph—mapping disease prevalence by crop type, geography, and season—enabling proactive alert recommendations and dynamically weighted RAG retrieval.

Agricultural marketplace integration. A marketplace module linking sales records to the diary and crop management history would provide end-to-end farm-to-market traceability. Buyers could verify agronomic practice records and AI-diagnosed disease history as part of quality certification.

Multi-agent advisory architecture. Specialized agents for soil management, pest control, irrigation scheduling, and market timing, coordinated by an orchestrating agent, could improve response specificity through targeted context routing.

Local LLM fine-tuning. Dependence on general-purpose LLMs introduces quality degradation for localized Vietnamese agricultural terminology and regional cultivation practices. Fine-tuning Qwen-2.5-7B or Vistral-7B on a Vietnamese agricultural instruction dataset constructed from the knowledge base would reduce API dependency while improving domain-specific response quality.

XII. CONCLUSION

This paper presented the design and prototype architecture of FarmDiaries AI, an AI-assisted agricultural diary and advisory system targeting Vietnamese smallholder farmers. The central technical contribution is a Hybrid RAG architecture enabling personalized agronomic recommendations grounded

in both verified domain knowledge and the individual farmer's operational history.

The architecture contributes four implementation-oriented design elements. First, it separates LLM invocation, prompt construction, retrieval, and embedding into independently testable modules. Second, it treats prompt construction as deterministic, versioned code while keeping authorization and output validation in the application layer. Third, it introduces an image-validation and safety pipeline using file verification, quality checks, duplicate detection, structured output, and regulatory cross-checking. Fourth, it defines a cost-constrained deployment strategy based on configurable quotas, asynchronous scheduling, and a rebuildable pgvector index. These are design contributions; their effectiveness, cost, latency, and operational reliability must be established through the evaluation protocol described in this paper.

The optional Farm Snap contribution workflow may support a later research phase by collecting explicitly consented, de-identified, quality-reviewed Vietnamese field-condition images. Whether this dataset is sufficient to improve a fine-tuned vision model must be tested rather than assumed. FarmDiaries AI offers a testable blueprint for studying personalized agricultural assistance under the connectivity, cost, privacy, and usability constraints of smallholder farming communities.

Acknowledgment

The authors thank the Faculty of Software Engineering at FPT University for supporting the SDN392 Capstone Project. Any participating farmers, agronomists, or partner organizations will be acknowledged only after the corresponding study and consent procedures have been completed.

References

[1] General Statistics Office of Vietnam, "Statistical Yearbook of Vietnam 2023," Statistical Publishing House, Hanoi, 2023.

[2] World Bank, "Vietnam Poverty and Equity Brief," The World Bank Group, Washington, D.C., 2023.

[3] A. Kamilaris and F. X. Prenafeta-Boldú, "Deep learning in agriculture: A survey," *Computers and Electronics in Agriculture*, vol. 147, pp. 70–90, 2018.

[4] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-T. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474.

[5] S. P. Mohanty, D. P. Hughes, and M. Salathé, "Using deep learning for image-based plant disease detection," *Frontiers in Plant Science*, vol. 7, p. 1419, 2016.

[6] D. P. Hughes and M. Salathé, "An open access repository of images on plant health to enable the development of mobile disease diagnostics through machine learning and crowdsourcing," *arXiv preprint arXiv:1511.08060*, 2015.

[7] D. Singh, N. Jain, P. Jain, P. Kayal, S. Kumawat, and N. Batra, "PlantDoc: A dataset for visual plant disease detection," in *Proceedings of the 7th ACM IKDD CoDS and 25th COMAD*, 2020, pp. 249–253.

[8] M. Tan and Q. V. Le, "EfficientNet: Rethinking model scaling for convolutional neural networks," in *Proceedings of the 36th International Conference on Machine Learning (ICML)*, vol. 97, 2019, pp. 6105–6114.

[9] A. Howard, M. Sandler, G. Chu, L.-C. Chen, B. Chen, M. Tan, W. Wang, Y. Zhu, R. Pang, V. Vasudevan, Q. V. Le, and H. Adam, "Searching for MobileNetV3," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2019, pp. 1314–1324.

[10] R. Sharma, S. Kumar, and A. Gupta, "Retrieval-augmented generation for agricultural advisory systems: A comparative study," in *Proceedings of the International Conference on AI in Agriculture*, 2023, pp. 112–121.

[11] C. Pinto, J. Alves, and T. Ferreira, "Large language models for crop advisory: A case study in Brazilian soybean cultivation," *Computers and Electronics in Agriculture*, vol. 208, p. 107765, 2023.

[12] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 779–788.

[13] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2019.

[14] M. G. Selvaraj, A. Vergara, H. Ruiz, N. Safari, S. Elayabalan, W. Ocimati, and G. Blomme, "AI-powered banana diseases and pest detection," *Plant Methods*, vol. 15, no. 1, pp. 1–11, 2019.

[15] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, A. Rodriguez, A. Joulin, E. Grave, and G. Lample, "LLaMA: Open and efficient foundation language models," *arXiv preprint arXiv:2302.13971*, 2023.